

PORTFOLIO PROJECT REPORT

ONLINE BOOK STORE DATABASE ANALYSIS

[Using SQL Server / T-SQL]

Prepared By:

TIYA PATEL

BSC in Information Technology (IT)

MAY 2026

1. PROJECT INTRODUCTION & OBJECTIVE

The "Online Book Store Management & Data Analysis" project is designed to manage and analyse a relational database system for a digital bookstore. The primary objective is to structured data relating to books, customers, and transactions to extract actionable business insights.

By implementing various SQL queries—ranging from basic filtering to advanced multi-table joins and data aggregations—this project simulates real-world business scenarios such as tracking sales performance, inventory control, and analysing customer purchasing behaviour.

Key Analytics Goals:

- **Business Optimization:** Identifying top-selling genres and authors to manage stock efficiently.
- **Customer Insights:** Tracking high-value customers and analysing geographical order distribution.
- **Inventory Management:** Monitoring stock levels and predicting remaining inventory post-order fulfilment.

2. DATABASE SCHEMA & ARCHITECTURE

The system utilizes an optimized relational model consisting of three core tables, interconnected via relational constraints to maintain high data integrity.

Core Database Tables:

1. **BOOKS:** Stores extensive details of books, including unique identifiers, title, genre, retail price, and available stock units.
2. **CUSTOMERS:** Contains profile records of registered buyers, capturing customer names, unique IDs, and country locations.
3. **ORDERS:** Captures transaction history including individual order IDs, customer relationships, order dates, purchase quantity, and total transaction amount.

Relational Constraints Implemented:

- **Primary Key (PK):** Set on Customers (Customer_ID) and Books (Book_ID) to guarantee unique record identification.
- **Foreign Key (FK):** Set on Orders (Customer_ID) referencing Customers (Customer_ID) to preserve strict referential integrity across sales transactions.

3. ANALYTICAL APPROACH & METHODOLOGY

The analysis is structured into two comprehensive stages to demonstrate full data manipulation capabilities:

• Phase 1: Basic & Intermediate Queries

Focuses on foundational data retrieval, date-range filtering, price/stock ranking (using TOP functions), and distinct value identification.

• Phase 2: Advanced Relational Analytics

Utilizes robust SQL features including INNER JOINS, LEFT JOINS for unmatched records, conditional aggregation with COALESCE (), group filtering using HAVING, and independent Correlated Subqueries.

Basic Queries

-- 1) Retrieve all books in the "Fiction" genre:

```
SELECT *
FROM Books
WHERE Genre='Fiction';
```

Query Output:

Results Messages							
	Book_ID	Title	Author	Genre	Published_Year	Price	Stock
1	4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.5200004577637	8
2	22	Multi-layered optimizing migration	Wesley Escobar	Fiction	1908	39.2299995422363	78
3	28	Expanded analyzing portal	Lisa Coffey	Fiction	1941	37.5099983215332	79
4	29	Quality-focused multi-tasking challenge	Katrina Underwood	Fiction	1905	31.1200008392334	100
5	31	Implemented encompassing conglomeration	Melissa Taylor	Fiction	2010	21.2299995422363	44
6	39	Optimized national process improvement	Megan Goodwin	Fiction	1978	10.9899997711182	42
7	40	Adaptive didactic interface	Natalie Gonzalez	Fiction	1923	25.9699993133545	94
8	47	Reverse-engineered directional conglomeration	John Christian	Fiction	2006	20.3700008392334	90
9	62	Re-contextualized real-time strategy	Nicole Lynch	Fiction	1953	26.3400001525879	23
10	63	Polarized heuristic database	Franklin Mack	Fiction	1989	22.3799991607666	56
11	100	Synchronized client-server service-desk	James Alvarado	Fiction	1906	49.8899993896484	29
12	116	Multi-tiered foreground contingency	Jamie Gates	Fiction	1938	41.8199996948242	50
13	125	Public-key analyzing Graphic Interface	Abigail Madden	Fiction	1990	32.4099998474121	16
14	130	Realigned context-sensitive pricing structure	Jason Rodriguez	Fiction	2004	6.6399998664856	90
15	134	Polarized bandwidth-monitored throughput	Linda Newman	Fiction	1955	35.7200012207031	49
16	142	Multi-tiered responsive parallelism	Amanda Wilson	Fiction	1940	48.9599990844727	11

-- 2) Find books published after the year 1950:

```
SELECT Book_ID,Title,Published_Year
FROM Books
WHERE Published_Year>1950;
```

Query Output:

Results Messages			
	Book_ID	Title	Published_Year
1	2	Persevering reciprocal knowledge user	1971
2	4	Customizable 24hour product	2020
3	5	Adaptive 5thgeneration encoding	1956
4	6	Advanced encompassing implementation	1985
5	8	Persistent local encoding	2019
6	9	Optimized interactive challenge	1987
7	10	Ergonomic national hub	2015
8	11	Secured zero tolerance time-frame	1998
9	12	Polarized optimal array	1989
10	15	User-friendly motivating strategy	1997
11	17	Reduced secondary core	1966
12	18	Adaptive 4thgeneration concept	2021
13	19	Progressive asymmetric Internet solution	1990
14	20	Face-to-face systematic throughput	1978
15	23	Reverse-engineered context-sensitive hardware	1967
16	25	Devolved mobile conglomeration	1984

-- 3) List all customers from the Canada:

```
SELECT * FROM Customers
WHERE Country='Canada';
```

Query Output:

	Customer_ID	Name	Email	Phone	City	Country
1	38	Nicholas Harris	christine93@perkins.com	1234567928	Davistown	Canada
2	415	James Ramirez	robert54@hall.com	1234568305	Maxwelltown	Canada
3	468	David Hart	stokesrebecca@gmail.com	1234568358	Thompsonfurt	Canada

-- 4) Show orders placed in November 2023:

```
SELECT * FROM Orders
WHERE Order_Date BETWEEN '2023-11-01' AND '2023-11-30';
```

Query Output:

	Order_ID	Customer_ID	Book_ID	Order_Date	Quantity	Total_Amount
1	4	433	343	2023-11-25	7	301.209991455078
2	19	496	60	2023-11-17	9	316.260009765625
3	75	291	375	2023-11-30	5	170.75
4	132	469	333	2023-11-22	7	194.320007324219
5	137	474	471	2023-11-25	8	363.040008544922
6	163	207	384	2023-11-23	3	101.76000213623
7	182	129	293	2023-11-01	7	125.51000213623
8	200	313	303	2023-11-23	1	6.57000017166138
9	213	325	447	2023-11-17	7	253.75
10	231	22	384	2023-11-11	1	33.9199981689453
11	245	386	97	2023-11-01	9	411.660003662109
12	252	405	387	2023-11-15	5	237.100006103516
13	257	123	403	2023-11-06	1	15.0100002288818
14	288	6	128	2023-11-13	1	24.0400009155273
15	307	368	133	2023-11-17	1	20.9599990844727
16	322	270	112	2023-11-08	2	16.0400009155273

-- 5) Retrieve the total stock of books available:

```
SELECT SUM(Stock) AS total_stock
FROM Books;
```

Query Output:

	total_stock
1	25056

-- 6) Find the details of the most expensive book:

```
SELECT TOP 1 *  
FROM Books  
ORDER BY Price DESC;
```

Query Output:

Results		Messages					
	Book_ID	Title	Author	Genre	Published_Year	Price	Stock
1	340	Proactive system-worthy orchestration	Robert Scott	Mystery	1907	49.9799995422363	88

-- 7) Show all customers who ordered more than 1 quantity of a book:

```
SELECT * FROM Orders  
WHERE Quantity>1;
```

Query Output:

Results		Messages				
	Order_ID	Customer_ID	Book_ID	Order_Date	Quantity	Total_Amount
1	1	84	169	2023-05-26	8	188.559997558594
2	2	137	301	2023-01-23	10	216.600006103516
3	3	216	261	2024-05-27	6	85.5
4	4	433	343	2023-11-25	7	301.209991455078
5	5	14	431	2023-07-26	7	136.360000610352
6	6	439	119	2024-10-11	5	249.399993896484
7	7	195	467	2023-10-23	6	82.9199981689453
8	8	32	159	2024-05-07	4	144.839996337891
9	9	109	407	2024-01-04	9	379.709991455078
10	10	94	122	2024-07-09	4	123
11	12	454	3	2024-06-17	2	31.5
12	13	420	180	2023-06-08	5	125.449996948242
13	14	454	319	2023-08-24	2	85.2200012207031
14	15	127	479	2023-01-10	6	229.619995117188
15	16	412	196	2023-10-06	8	53.5200004577637
16	17	462	481	2023-03-20	5	52.75

-- 8) Retrieve all orders where the total amount exceeds \$20:

```
SELECT * FROM Orders  
WHERE Total_Amount>20;
```

Query Output:

	Order_ID	Customer_ID	Book_ID	Order_Date	Quantity	Total_Amount
1	1	84	169	2023-05-26	8	188.559997558594
2	2	137	301	2023-01-23	10	216.600006103516
3	3	216	261	2024-05-27	6	85.5
4	4	433	343	2023-11-25	7	301.209991455078
5	5	14	431	2023-07-26	7	136.360000610352
6	6	439	119	2024-10-11	5	249.399993896484
7	7	195	467	2023-10-23	6	82.9199981689453
8	8	32	159	2024-05-07	4	144.839996337891
9	9	109	407	2024-01-04	9	379.709991455078
10	10	94	122	2024-07-09	4	123
11	11	131	206	2023-10-16	1	38.0099983215332
12	12	454	3	2024-06-17	2	31.5
13	13	420	180	2023-06-08	5	125.449996948242
14	14	454	319	2023-08-24	2	85.2200012207031
15	15	127	479	2023-01-10	6	229.619995117188
16	16	412	196	2023-10-06	8	53.5200004577637

-- 9) List all genres available in the Books table:

```
SELECT DISTINCT genre FROM Books;
```

Query Output:

	genre
1	Biography
2	Fantasy
3	Fiction
4	Mystery
5	Non-Fiction
6	Romance
7	Science Fiction

-- 10) Find the book with the lowest stock:

```
SELECT TOP 1 *
FROM Books
ORDER BY Stock ;
```

Query Output:

	Book_ID	Title	Author	Genre	Published_Year	Price	Stock
1	44	Networked systemic implementation	Ryan Frank	Science Fiction	1965	13.5500001907349	0

-- 11) Calculate the total revenue generated from all orders:

```
SELECT SUM(total_amount)
AS Revenue
FROM Orders;
```

Query Output:

Results		Messages	
	Revenue		
1	75628.6601614952		

Advance Queries:

-- 1) Retrieve the total number of books sold for each genre:

```
SELECT b.Genre, SUM(O.Quantity) AS Total_Book_Sold
FROM Orders O
JOIN Books b ON o.Book_ID = b.Book_ID
GROUP BY b.Genre;
```

Query Output:

Results		Messages	
	Genre	Total_Book_Sold	
1	Biography	285	
2	Fantasy	446	
3	Fiction	225	
4	Mystery	504	
5	Non-Fiction	351	
6	Romance	439	
7	Science Fiction	447	

-- 2) Find the average price of books in the "Fantasy" genre:

```
SELECT AVG(Price) AS Average_price
FROM Books
WHERE GENRE = 'Fantasy';
```

Query Output:

Results		Messages	
	Average_price		
1	25.9816901112946		

-- 3) List customers who have placed at least 2 orders:

```
SELECT c.Name,o.Customer_ID, COUNT(o.Order_ID) AS Order_Count
FROM Orders O
JOIN Customers c ON o.Customer_ID = c.Customer_ID
GROUP BY o.Customer_ID, c.Name
HAVING COUNT(Order_ID) >= 2;
```

Query Output:

Results		Messages	
	Name	Customer_ID	Order_Count
1	Crystal Clements	2	2
2	Stephen Vasquez	6	2
3	Matthew Johnson	8	2
4	Kristine Kim	13	2
5	John Wood	14	2
6	Vanessa Gaines	15	2
7	Stacey Flores	16	3
8	Edgar Frost	21	2
9	Stacey Adams	22	3
10	Hannah Drake	23	2
11	Christina Mitchell	24	2
12	Ryan Lara	37	2
13	Debra Park	47	3

-- 4) Find the most frequently ordered book:

```
SELECT TOP 1 o.Book_ID,b.Title, COUNT(o.order_ID) AS ORDER_COUNT
FROM Orders o
JOIN books b ON o.Book_ID = b.Book_ID
GROUP BY o.Book_ID, b.Title
ORDER BY ORDER_COUNT DESC;
```

Query Output:

Results		Messages	
	Book_ID	Title	ORDER_COUNT
1	333	Advanced responsive extranet	4

-- 5) Show the top 3 most expensive books of 'Fantasy' Genre :

```
SELECT TOP 3 *
FROM Books
WHERE Genre = 'Fantasy'
ORDER BY Price DESC;
```

Query Output:

Results		Messages					
	Book_ID	Title	Author	Genre	Published_Year	Price	Stock
1	240	Stand-alone content-based hub	Lisa Ellis	Fantasy	1957	49.9000015258789	41
2	462	Innovative 3rdgeneration database	Allison Contreras	Fantasy	1988	49.2299995422363	62
3	238	Optimized even-keeled analyzer	Sherri Griffith	Fantasy	1975	48.9700012207031	72

-- 6) Retrieve the total quantity of books sold by each author:

```
SELECT b.Author,SUM(o.Quantity) AS Total_Quantity
from Orders o
JOIN Books b ON o.Book_ID = b.Book_ID
GROUP BY b.Author;
```

Query Output:

Results		Messages
	Author	Total_Quantity
1	Aaron Hall Jr.	4
2	Aaron Rangel	6
3	Abigail Madden	5
4	Adrian Avery	6
5	Alexander Bailey	6
6	Alexander Smith	7
7	Alexander Wallace	20
8	Alexandra Cox	8
9	Allison Contreras	6
10	Alyssa Lopez	5
11	Amanda Carrillo	7
12	Amanda Knight	10
13	Amanda Moore	14

-- 7) List the cities where customers who spent over \$30 are located:

```
SELECT DISTINCT c.city,Total_Amount
FROM Orders o
JOIN Customers c ON o.Customer_ID = c.Customer_ID
WHERE o.Total_Amount >= 30;
```

Query Output:

Results		Messages
	city	Total_Amount
1	Adkinsview	125.51000213623
2	Adrianafort	79.2900009155273
3	Aguilarside	148.679992675781
4	Aguilarside	148.770004272461
5	Aguilarside	253.75
6	Alanton	38.6399993896484
7	Alanton	170.75
8	Amandamouth	49.7200012207031
9	Angelaside	42.189998626709
10	Angelastad	426.100006103516
11	Angelatown	87.0100021362305
12	Annhaven	151.199996948242
13	Annhaven	156.869995117188

-- 8) Find the customer who spent the most on orders:

```
SELECT TOP 1 c.Customer_ID,c.name,SUM(o.Total_Amount) AS Total_Spent
FROM Orders o
JOIN Customers c ON o.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID,c.name
ORDER BY Total_Spent DESC;
```

Query Output:

Results

Messages

	Customer_ID	name	Total_Spent
1	457	Kim Turner	1398.90000915527

--9) Calculate the stock remaining after fulfilling all orders:

```
SELECT
b.Book_ID,
b.Title,
b.Stock,
COALESCE(SUM(o.Quantity),0) AS Order_Quantity,
b.Stock - COALESCE(SUM(o.Quantity),0) AS Remaining_Quantity
FROM Books b
LEFT JOIN Orders o ON b.Book_ID = o.Book_ID
GROUP BY
b.Book_ID,
b.Title,
b.Stock;
```

Query Output:

 Results  Messages					
	Book_ID	Title	Stock	Order_Quantity	Remaining_Quantity
1	1	Configurable modular throughput	100	3	97
2	2	Persevering reciprocal knowledge user	19	0	19
3	3	Streamlined coherent initiative	27	5	22
4	4	Customizable 24hour product	8	0	8
5	5	Adaptive 5thgeneration encoding	16	8	8
6	6	Advanced encompassing implementation	2	0	2
7	7	Open-architected exuding structure	95	5	90
8	8	Persistent local encoding	84	3	81
9	9	Optimized interactive challenge	70	0	70
10	10	Ergonomic national hub	25	1	24
11	11	Secured zero tolerance time-frame	10	5	5
12	12	Polarized optimal array	63	0	63
13	13	Adaptive 5thgeneration orchestration	99	9	90